

Dezembro 2025

# SISTEMA GRÁFICO ITERATIVO 2D

Gabriela e Eduardo Bif

# FEATURES

*conter/manipular as seguintes:*

- Manipular o formato de dados wavefront object (leitura de mais de um objeto ao mesmo tempo)
- Transformação de viewport
- Organização dos objetos em estruturas (p.ex., Winged-edge ou Half-edge)
- Transformação dos objetos (translação, rotação, cisalhamento e escalamento)
- Clipping antes do redesenho via algoritmo explícito (p.ex., Bresenham ou Xiaolin Wu)



## Manipular o formato de dados wavefront object (leitura de mais de um objeto ao mesmo tempo)

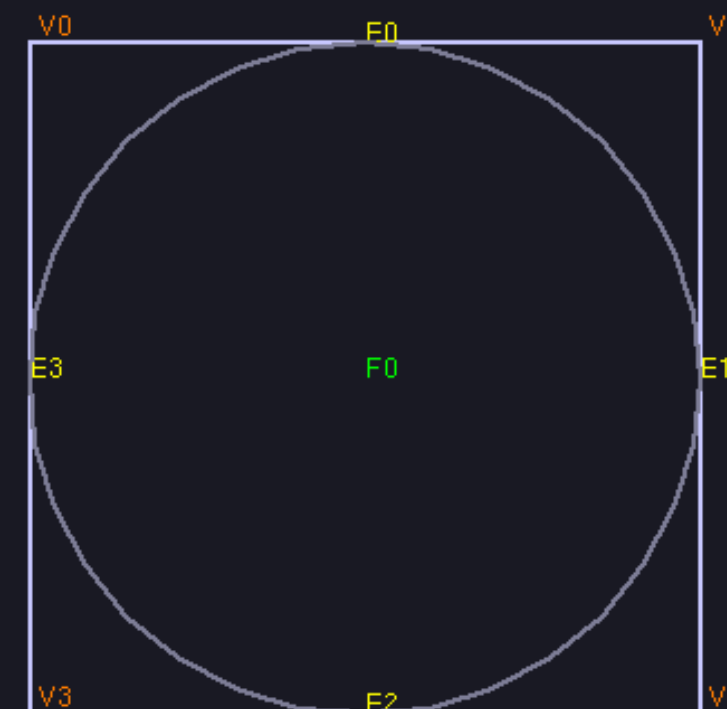
A função `loadOBJ` (processa o arquivo OBJ) e as variáveis globais - arrays - `g_objects` (armazena cada arquivo OBJ lido como uma entidade gráfica separada) e `g_activeObject` (referencia o objeto escolhido) permitem carregar e alternar entre vários arquivos OBJ (controle 'N').

```
Total de Objetos Carregados: 2  
Objeto ATIVO: ./cubo_1.obj
```

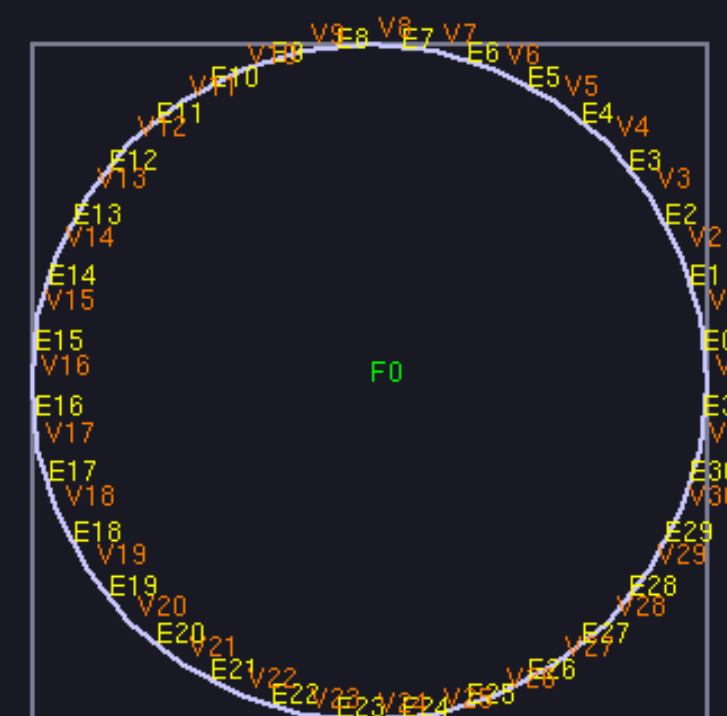
```
>>> Objeto ATIVO alterado para: ./cubo_1.obj
```

```
>>> Objeto ATIVO alterado para: ./esfera_2.obj
```

OBJETO ATIVO: ./cubo\_1.obj



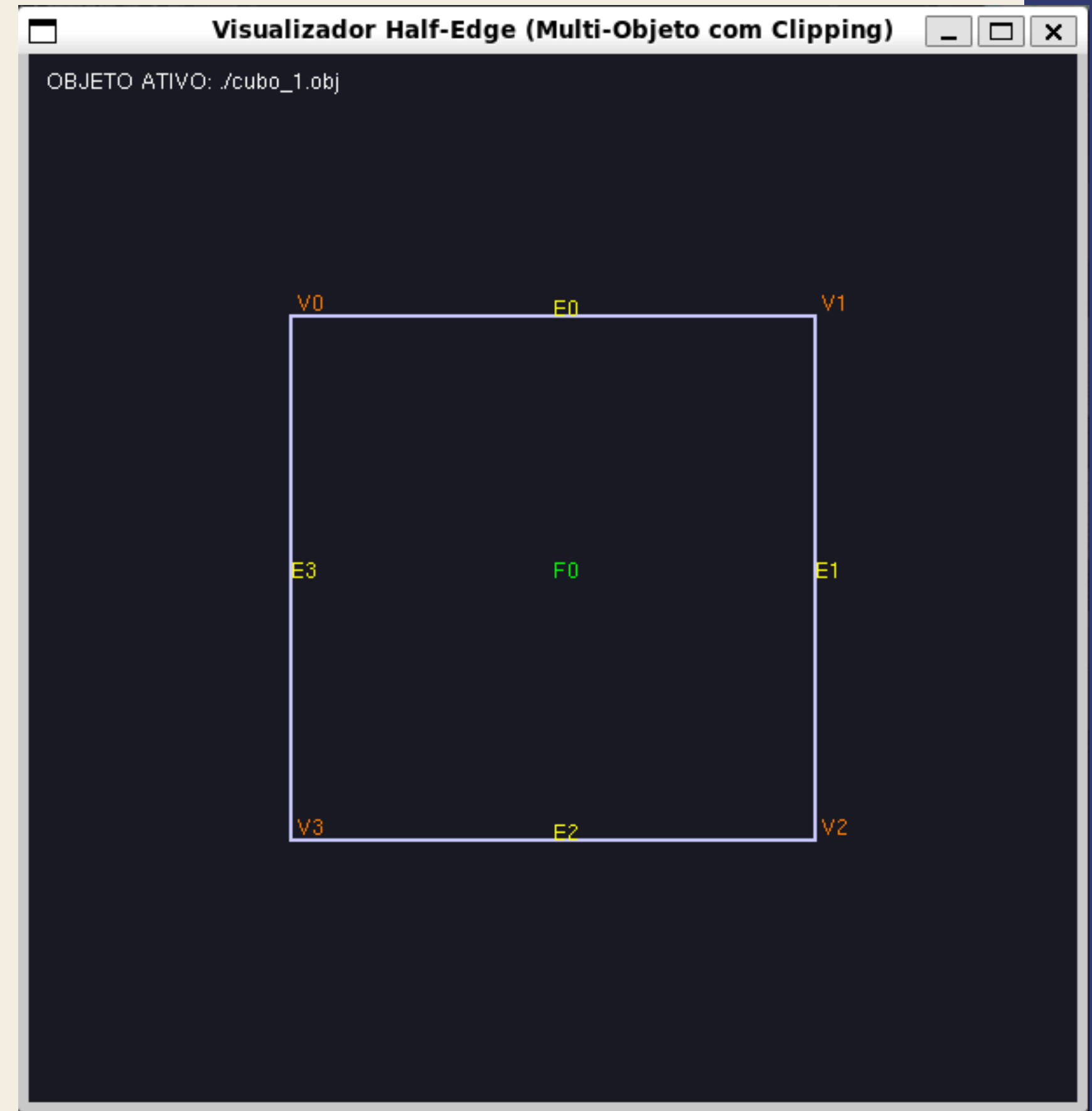
OBJETO ATIVO: ./esfera\_2.obj



## Transformação de viewport

A função **worldToPixel** faz o mapeamento refinado de coordenadas de mundo (WCS)  $[-1, 1]$  - que é o escopo da janela de clipping- para coordenadas de tela/pixel da resolução da janela OpenGL(DCS)  $[0, W] \times [0, H]$  através de **Normalização**.

```
--- Janela Grafica Iniciada ---  
  
--- Pressione as teclas na JANELA GRAFICA para interagir ---  
--- Controles de Objeto ---  
[N] - Alternar para o Proximo Objeto Ativo  
[V] [E] [F] - Ligar/Desliga IDs (Objeto Ativo)
```





## Organização Estrutural (Half-Edge)

A Estrutura de Dados Half-Edge é o núcleo topológico do sistema, fornecendo uma representação eficiente das faces, arestas e vértices, e suas relações de vizinhança. Maneira eficiente de armazenar e acessar informações de vizinhança em uma malha.

O sistema utiliza as **Estruturas(structs) Vertex, HalfEdge, e Face** são usadas para implementar a **topologia do objeto 2D Half-Edge** (função buildHalfEdgeStructure).

## Transformações de Objeto

Funções `applyTranslation`, `applyScaling`, `applyRotation`, `applyShearing`, e `applyReflection` implementam as transformações 2D, aplicadas ao objeto ativo (controle 'T', 'S', 'R', 'H', 'M').

```
--- Teclas de Transformacao (entrada no terminal - Objeto Ativo) ---  
[T] - Translação (dx dy)  
[S] - Escala (sx sy)  
[R] - Rotação (ângulo)  
[H] - Cisalhamento (shx shy)  
[M] - Reflexão (eixo 'x' ou 'y')  
  
--- Outros Controles ---  
[W] - Salvar estado do Objeto Ativo para 'output.obj'  
[ESC] - Fechar o programa.
```

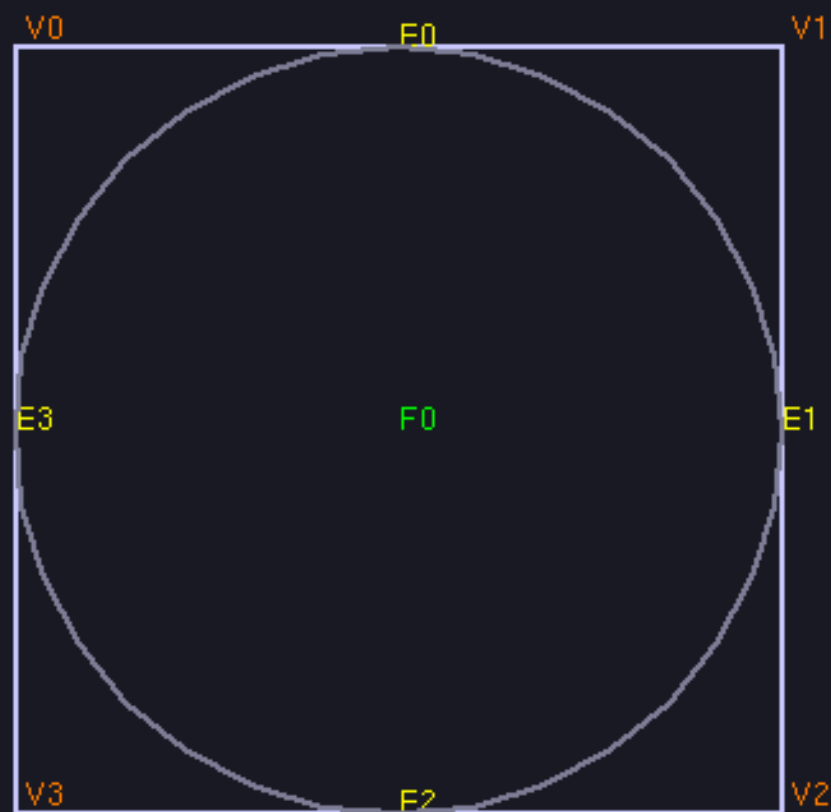
# Transformações de Objeto

## Translação

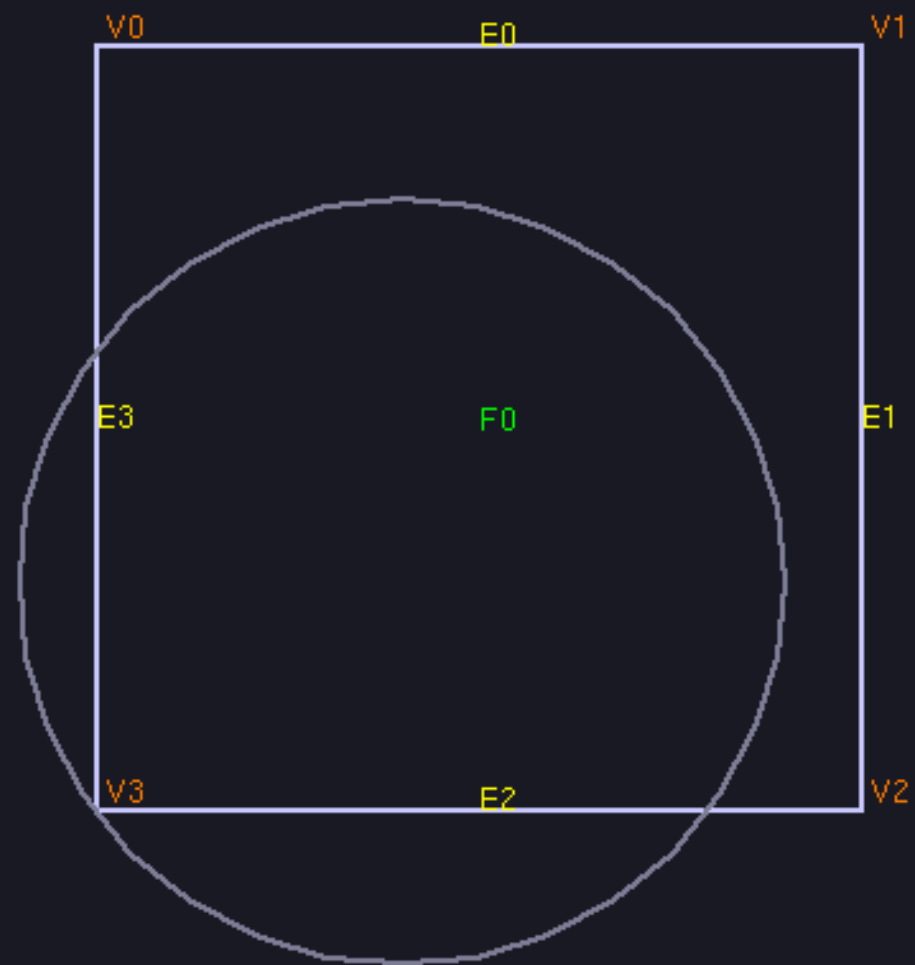
```
[TRANSFORMAÇÃO] Digite dx dy (ex: 0.1 0.0): 0.1 0.2  
> Aplicando Translação (0.1, 0.2) ao objeto: ./cubo_1.obj
```

```
[TRANSFORMAÇÃO] Digite dx dy (ex: 0.1 0.0): -0.1 -0.2  
> Aplicando Translação (-0.1, -0.2) ao objeto: ./cubo_1.obj
```

OBJETO ATIVO: ./cubo\_1.obj



OBJETO ATIVO: ./cubo\_1.obj

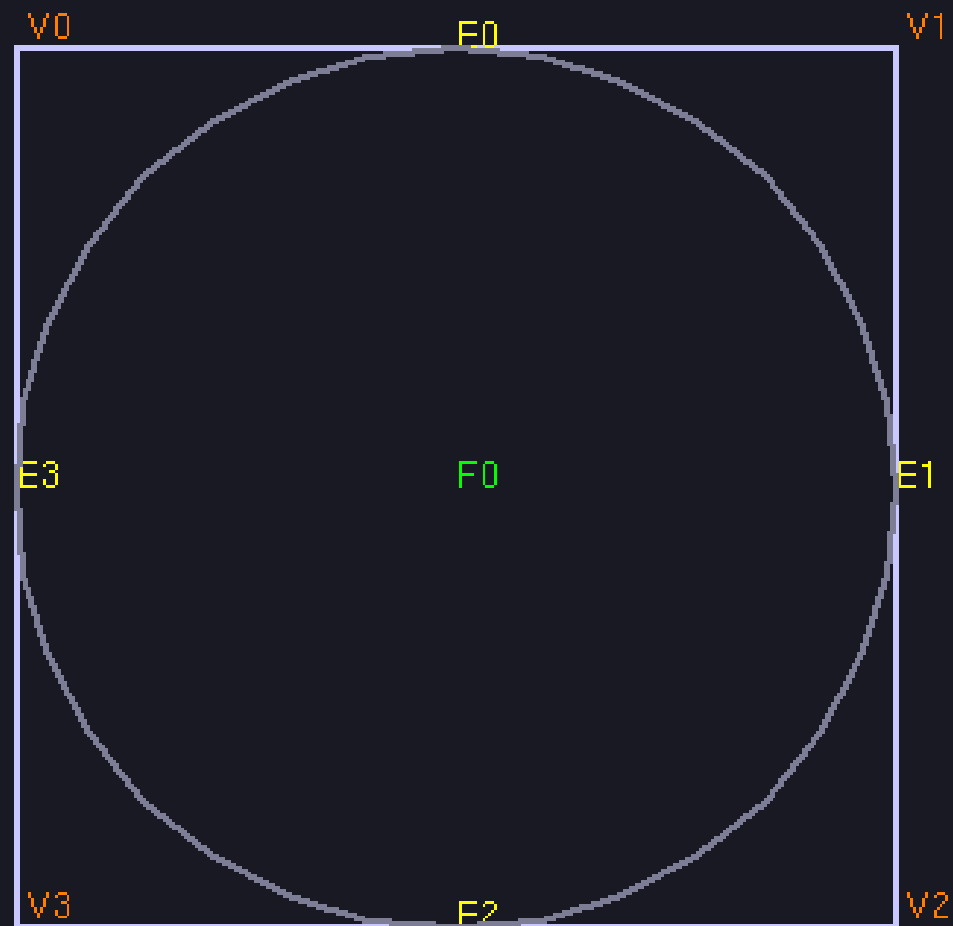


# Transformações de Objeto

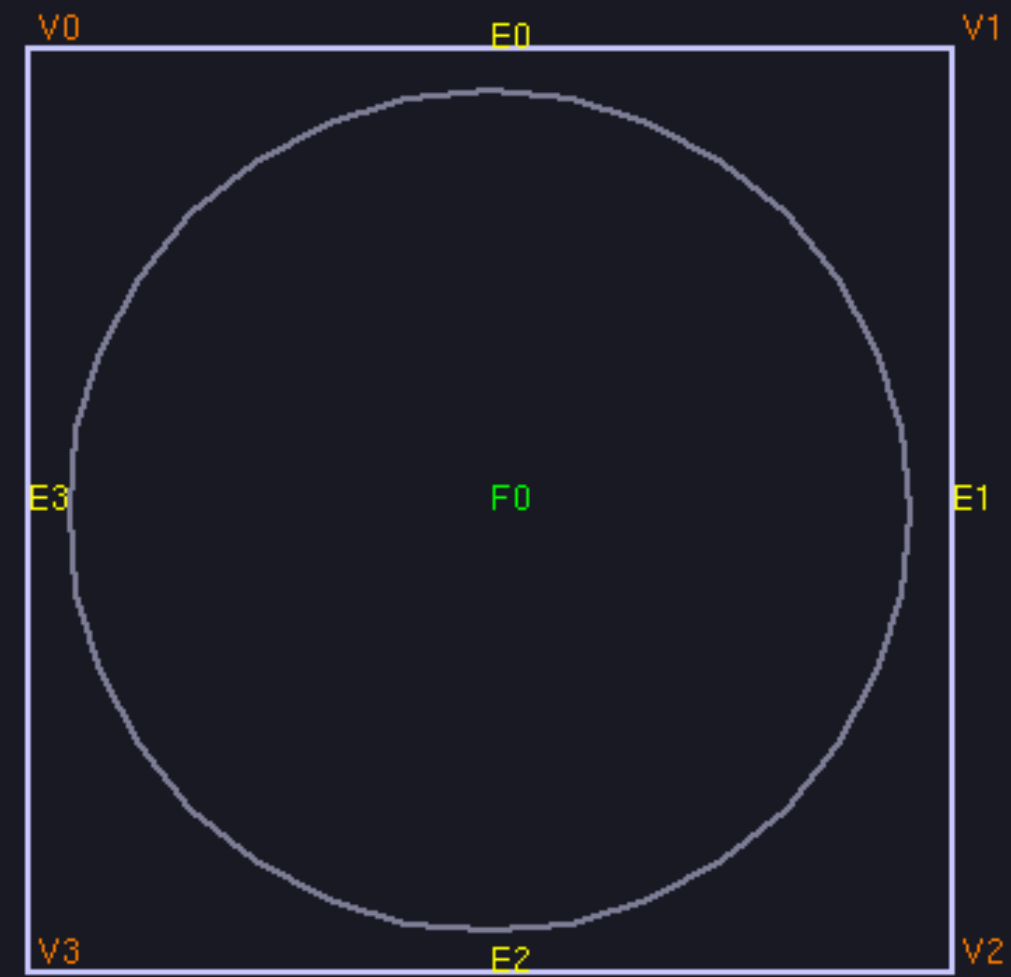
## Escala

```
[TRANSFORMAÇÃO] Digite sx sy (ex: 1.1 1.1): 1.1 1.1  
> Aplicando Escala (1.1, 1.1) ao objeto: ./cubo_1.obj
```

OBJETO ATIVO: ./cubo\_1.obj



OBJETO ATIVO: ./cubo\_1.obj

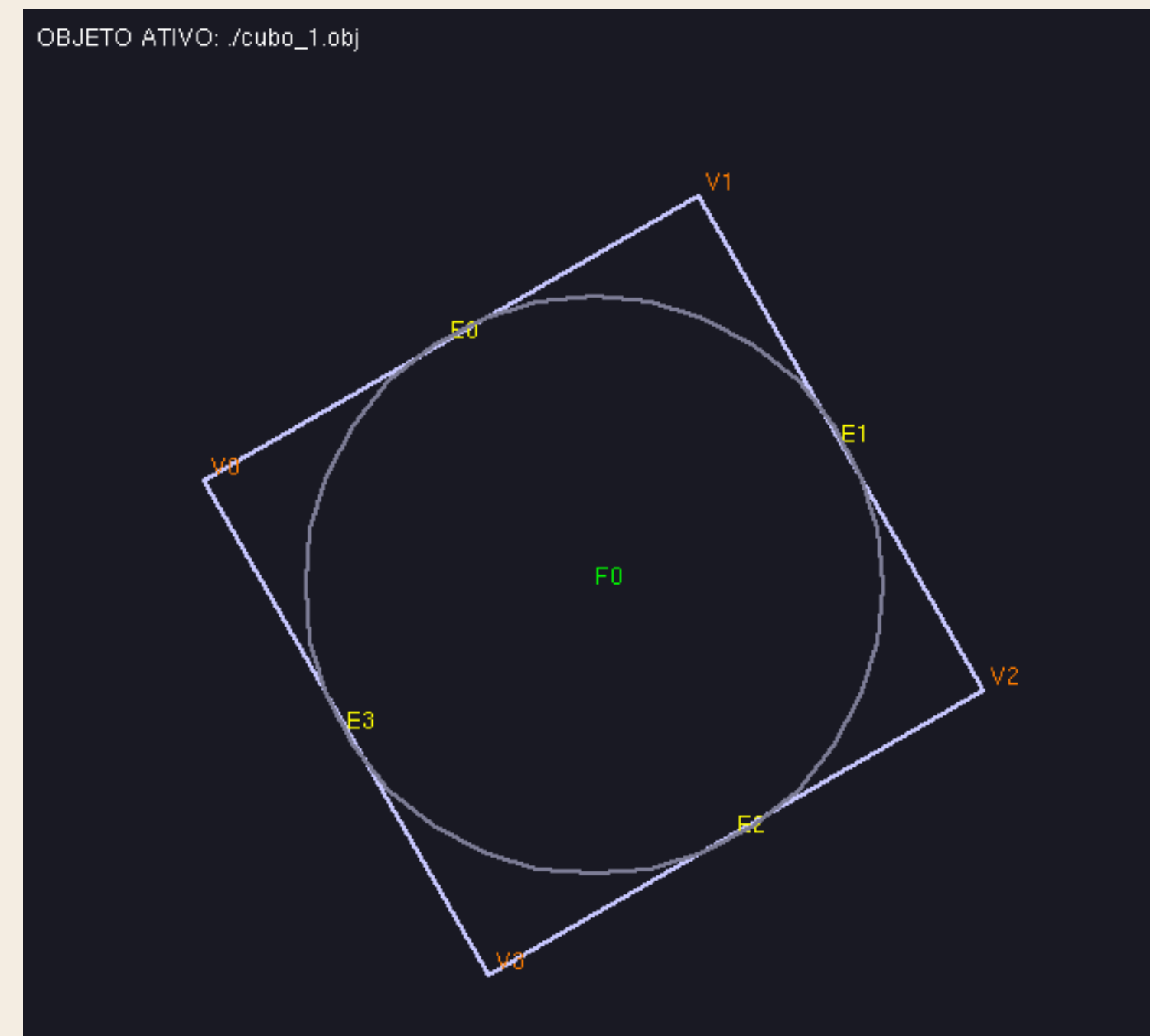
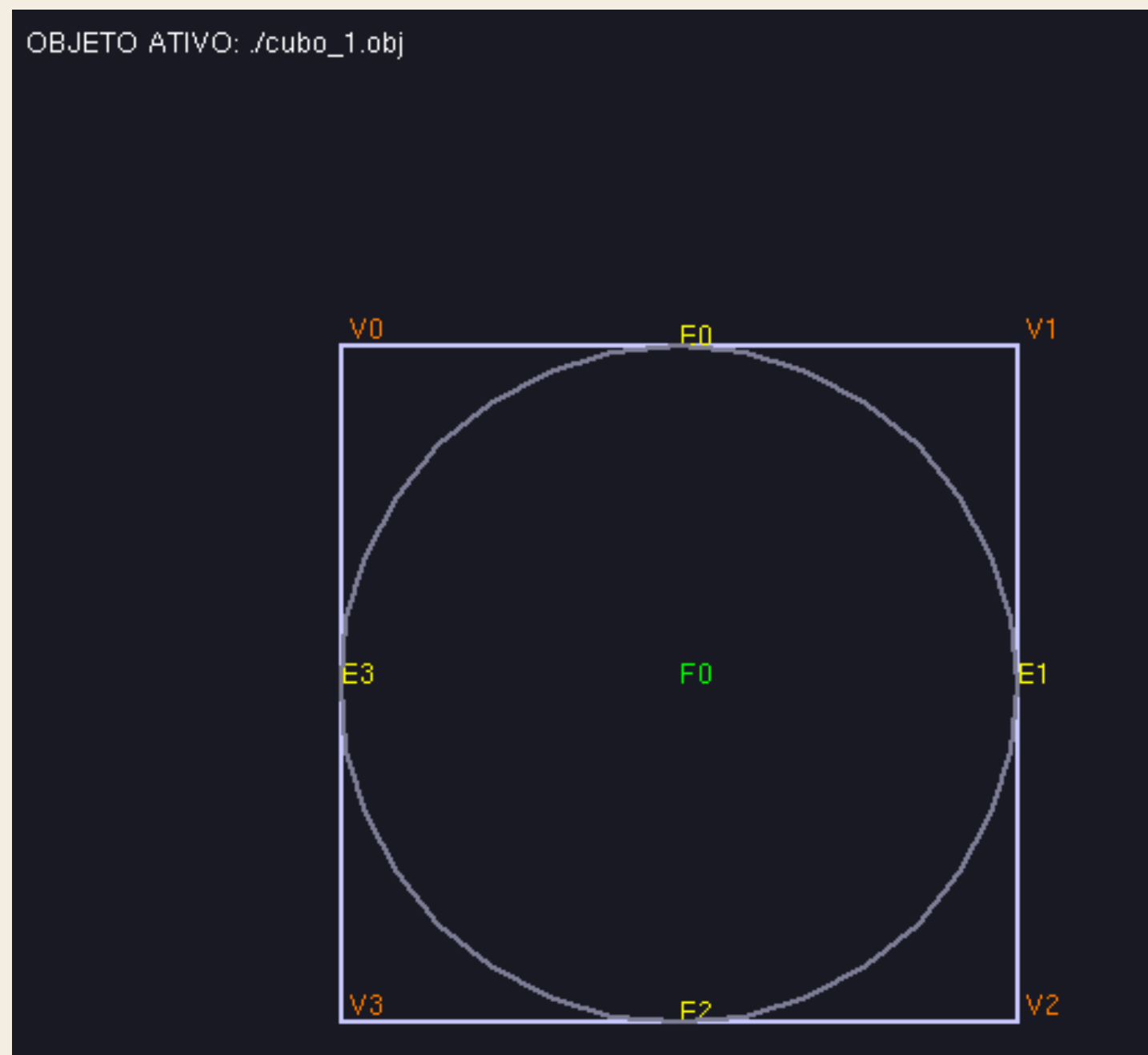




# Transformações de Objeto

## Rotação

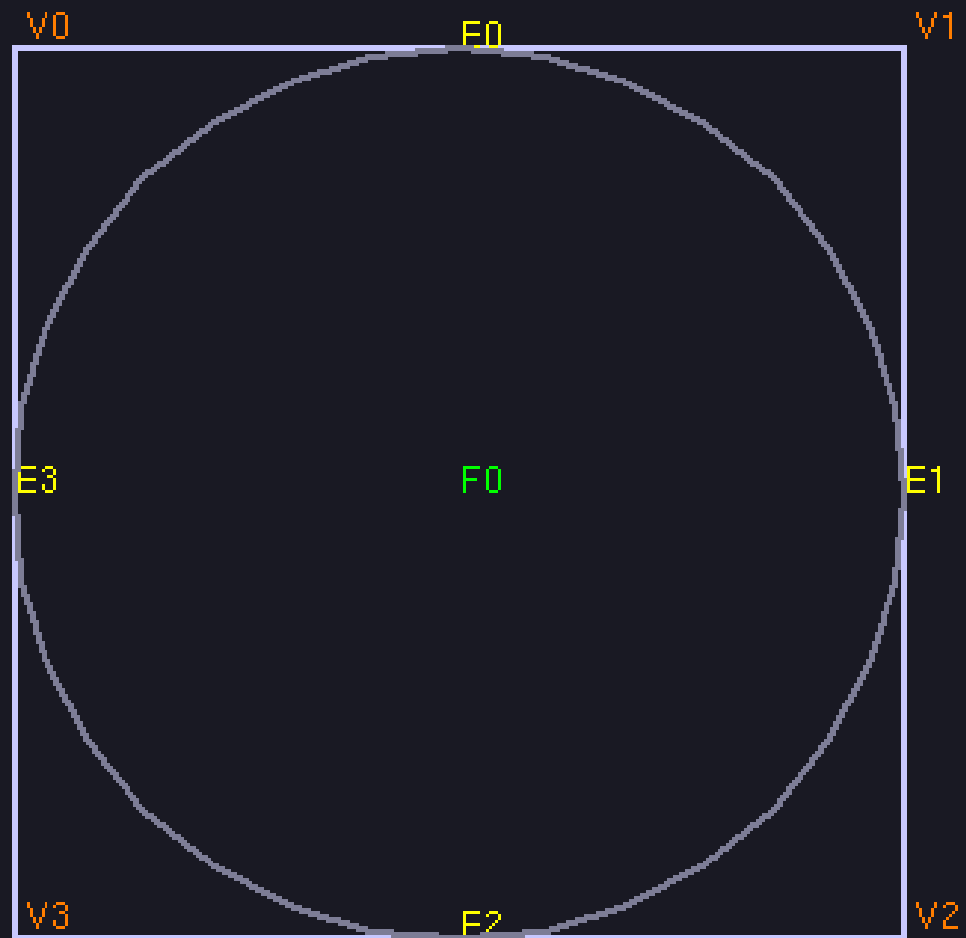
```
[TRANSFORMAÇÃO] Digite o angulo (ex: 15.0): 30  
> Aplicando Rotação (30 graus) ao objeto: ./cubo_1.obj
```



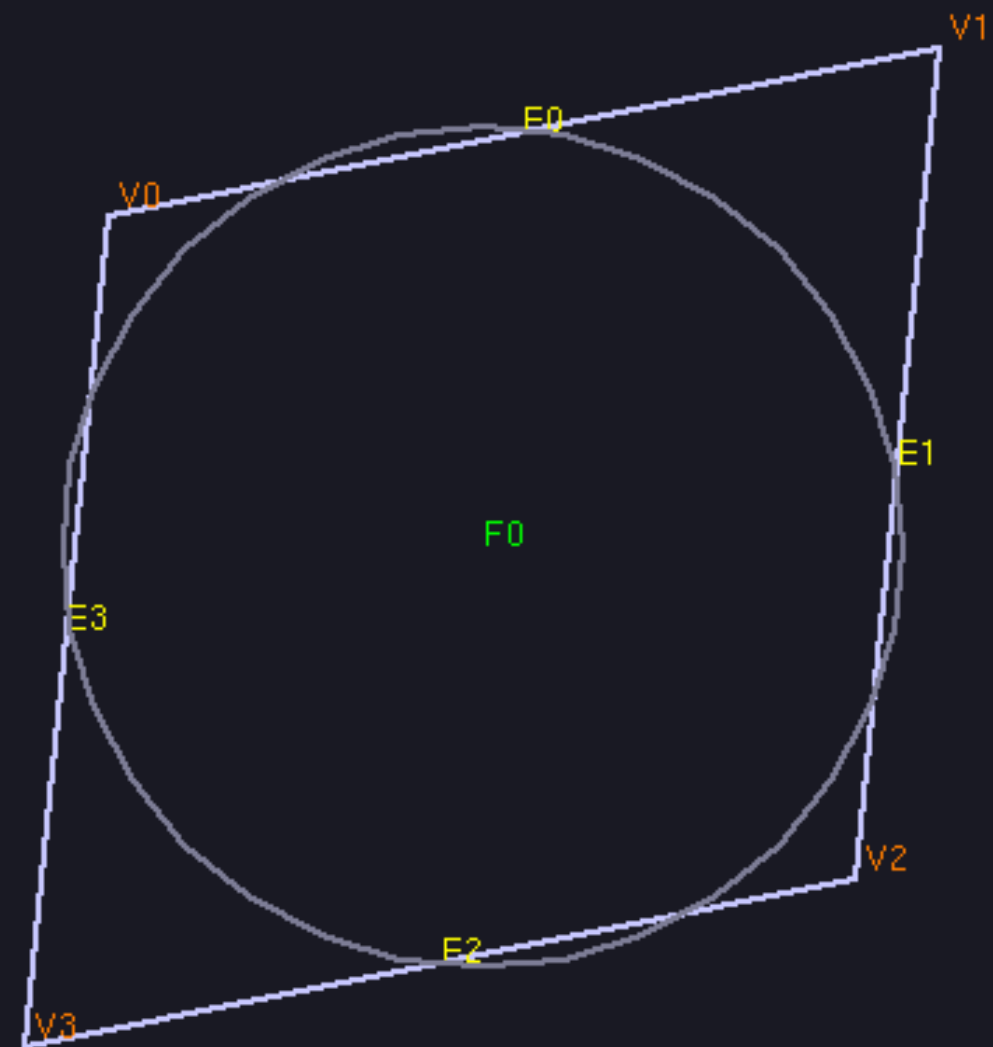
## Cisalhamento

```
[TRANSFORMAÇÃO] Digite shx shy (ex: 0.1 0.0): 0.1 0.2  
> Aplicando Cisalhamento (0.1, 0.2) ao objeto: ./cubo_1.obj
```

OBJETO ATIVO: ./cubo\_1.obj



OBJETO ATIVO: ./cubo\_1.obj

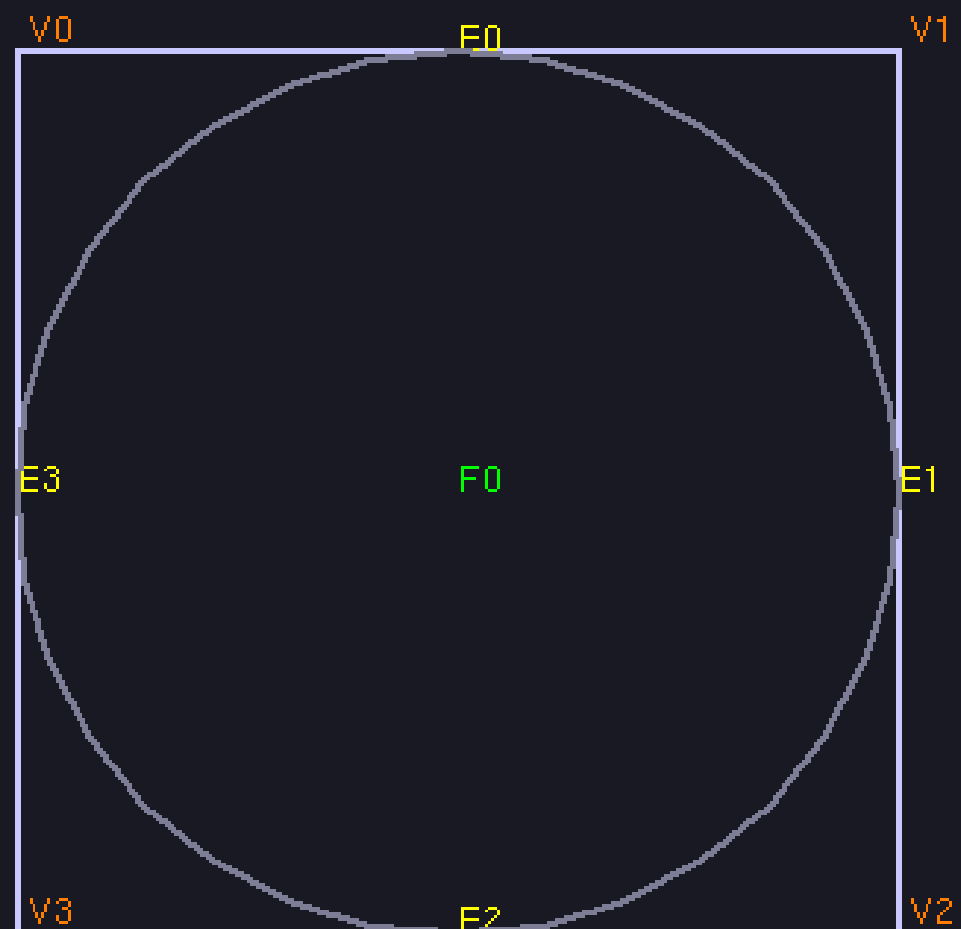


# Transformações de Objeto

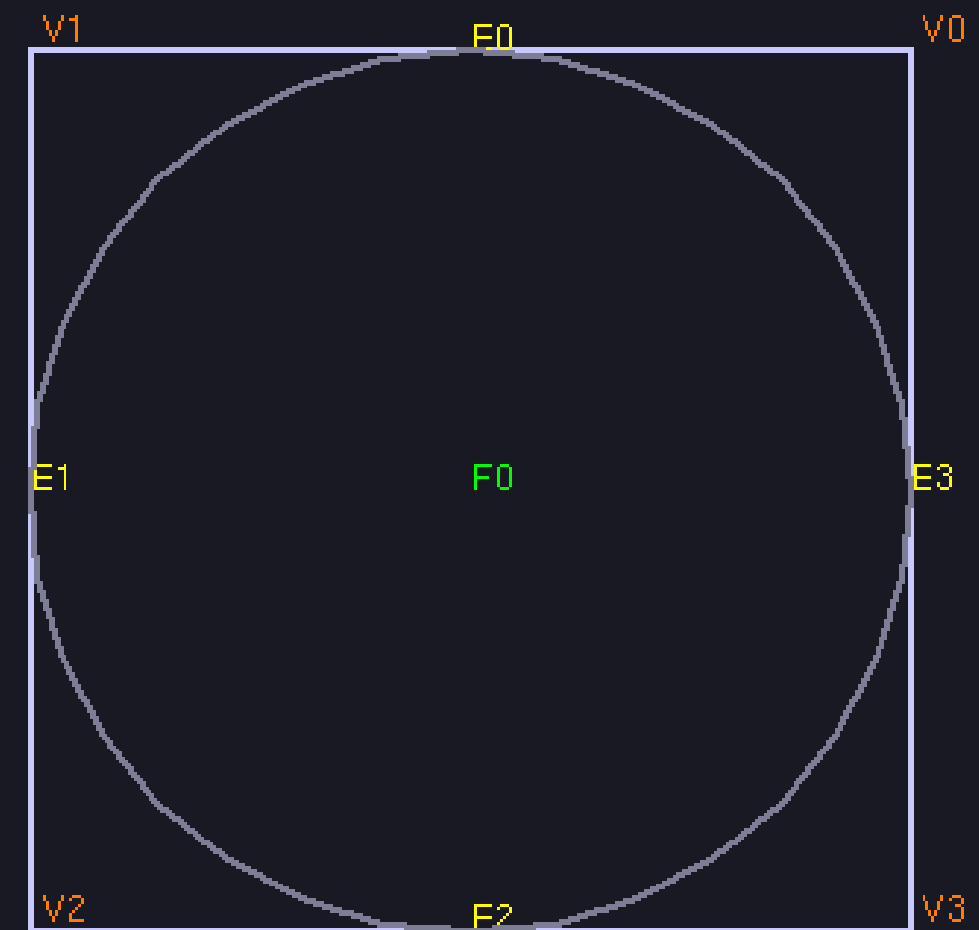
## Reflexão

```
[TRANSFORMAÇÃO] Digite o eixo (x ou y): x  
> Aplicando Reflexão (X=1, Y=0) ao objeto: ./cubo_1.obj
```

OBJETO ATIVO: ./cubo\_1.obj



OBJETO ATIVO: ./cubo\_1.obj



## Clipping antes do Redesenho

O algoritmo Cohen-Sutherland (`cohenSutherlandClip`) é aplicado diretamente nas coordenadas de mundo antes da conversão para pixel e antes da rasterização. Função `computeCode`.

## Rasterização Explícita

Suporte aos algoritmos Bresenham (`drawBresenham`) e Xiaolin Wu (`drawXiaolinWu`), selecionados na linha de comando.

```
./visualizador xiaolin_wu ./esfera_2.obj
```

```
./visualizador bresenham ./esfera_2.obj
```

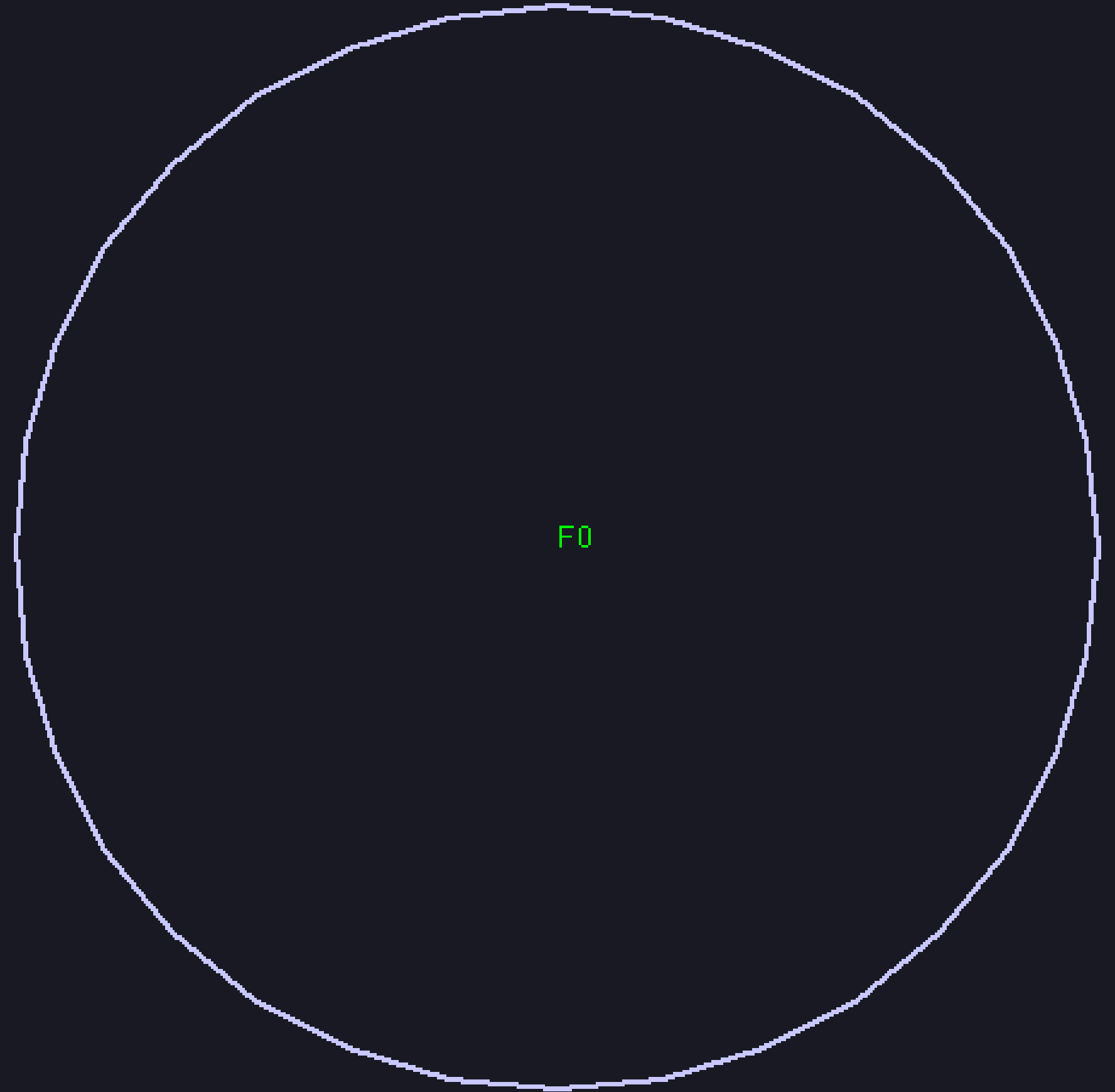


./visualizador *xiaolin\_wu* ./esfera\_2.obj

OBJETO ATIVO: ./esfera\_2.obj



./visualizador *bresenham* ./esfera\_2.obj





`./visualizador xiaolin_wu ./esfera_2.obj`

`./visualizador bresenham ./esfera_2.obj`





# OBRIGADO!

perguntas?

```
g++ main.cpp -o visualizador -lglut -lGL -lGLU -lpthread  
./visualizador bresenham output.obj  
Use 'bresenham' or 'xiaolin_wu'.
```